Meu Oráculo — Visão Geral do Sistema

> *Aplicação de finanças pessoais completa. React + TypeScript + Supabase + TailwindCSS.*

🏢 Funcionalidades do Produto

🏠 Dashboard

- Resumo financeiro do mês atual: saldo atual, saldo projetado, receitas e despesas
- Navegação entre meses (anterior / próximo)
- Alertas automáticos: contas vencendo hoje, contas em atraso, alerta de saldo negativo
- Meta mensal de saldo com barra de progresso e % atingida
- Top 5 categorias de gastos do mês
- Cartão de alertas do carro (km para próxima manutenção)

🛒 Compras (`/shopping`)

- **Lista de compras** com status `aberta` / `finalizada`
- Adicionar itens com **categorização automática** por nome (ex: "arroz" → "Grãos")
- Adicionar múltiplos itens de uma vez (modo bulk)
- Marcar itens como "pegado" no carrinho
- Editar quantidade, unidade e preço por item
- **Catálogo de produtos** aprendido das compras anteriores (autocomplete)
- **Duplicar lista** (cópia com todos os itens)
- Renomear e excluir listas
- **Finalizar lista**: registra automaticamente uma despesa no Caixa (categoria "Mercado")
- **Reabrir lista**: remove o lançamento do caixa correspondente
- Dashboard de compras: histórico de gastos, comparativo de meses, total por categoria
- Sugestões de itens baseadas no catálogo (IA de categorização)

🚗 Carro (`/car`)

- **Combustível**: registrar abastecimentos (km, valor, forma de pagamento)
  - Calcula automaticamente consumo médio (km/l)
  - Registra despesa no Caixa
- **Manutenções**: registrar revisões, troca de óleo, pneus etc.
  - Registra km realizado e km da próxima manutenção
  - Registra despesa no Caixa
- Alertas de manutenção: `OK` / `ATENÇÃO` / `URGENTE` baseado em km restantes
- Histórico completo de abastecimentos e manutenções

💰 Caixa (`/ledger`)

- Registro de receitas e despesas manuais
- Tipos: `Receita` ou `Despesa`
- Categorias, descrição, data e forma de pagamento
- Edição e exclusão de lançamentos
- Filtro por mês com navegação
- Resumo do mês: receitas, despesas e saldo
- Saldo de abertura acumulado de meses anteriores
- Meta mensal com porcentagem visual
- **Exclui automaticamente** entradas de faturas de cartão (evita dupla contagem)

📋 Contas Fixas (`/bills`)

- **Despesas Fixas** mensais (ex: aluguel, conta de luz)
  - Marcar como paga com data de pagamento real
  - Categoria e valor
- **Contas Recorrentes** (modelos mestres)
  - Configurar dia de vencimento (1-31)
  - Valor estimado (fixo ou variável — ex: água)
  - Ativar/desativar
  - **Geração mensal automática**: ao mudar de mês, o sistema cria as instâncias mensais
- **Faturas de Cartão de Crédito** (virtuais, geradas automaticamente)
  - Agregam todas as compras no cartão do ciclo
  - Aparecem como conta a pagar quando não pagas
- Status visual: `Paga` / `Hoje` / `Em atraso` / `A vencer`
- Valor pendente total do mês

💳 Cartões de Crédito (`/cards`)

- Cadastro de cartões com nome, cor, limite, dia de fechamento e dia de vencimento
- **Transações de cartão**: registrar compras com parcelamento (até 48x)
  - Cada parcela é criada como transação separada com data calculada automaticamente
- Editar e excluir transações
- Visualização de faturas por período
- Status de fatura: `Em aberto` / `Fatura fechada` / `Pago`
- Limite utilizado vs disponível por cartão

👤 Perfil (`/profile`)

- Visualização e edição de dados do usuário
- Tema: `Claro` / `Escuro`
- Configurações de acessibilidade
- Logout

📐 Arquitetura Técnica

Stack

Camada	Tecnologia
Frontend	React 19 + TypeScript + Vite 7
Estilo	TailwindCSS 3
Roteamento	React Router DOM 7
Backend/DB	Supabase (PostgreSQL + Auth + RLS)
Ícones	Lucide React
Testes	Vitest 4
Validação	Zod 4

Estrutura de Diretórios

```
src/
├── context/
│   ├── AppContext.tsx          ← Orquestrador (após refatoração)
│   ├── slices/
│   │   ├── ledgerSlice.ts      ← Domínio: Caixa
│   │   ├── carSlice.ts         ← Domínio: Carro
│   │   ├── shoppingSlice.ts    ← Domínio: Compras
│   │   ├── billsSlice.ts       ← Domínio: Contas
│   │   └── cardsSlice.ts       ← Domínio: Cartões
│   ├── AuthContext.tsx
│   ├── ToastContext.tsx         ← [NOVO] Notificações globais
│   ├── ThemeContext.tsx
│   └── AccessibilityContext.tsx
├── hooks/
│   ├── useFinancialSummary.ts  ← [NOVO] Wrapper do hook
│   └── __tests__/
│       └── financialCalculations.test.ts  ← [NOVO] 17 testes
├── lib/
│   ├── financialCalculations.ts ← [NOVO] Lógica pura testável
│   ├── supabaseService.ts
│   ├── supabaseClient.ts
│   ├── categorias.ts
│   ├── ledger-categories.ts
│   └── utils.ts                 ← [MODIFICADO] +getDueStatus()
├── schemas/                     ← [NOVO]
│   ├── ledger.schema.ts
│   ├── fixedExpense.schema.ts
│   ├── creditCard.schema.ts
│   └── __tests__/schemas.test.ts ← 23 testes
├── components/
│   ├── ErrorBoundary.tsx        ← [MODIFICADO] UI dark + recuperação
│   ├── ui/ToastContainer.tsx    ← [NOVO]
│   └── ...
├── pages/
│   ├── Dashboard.tsx            ← [MODIFICADO] usa useFinancialSummary
│   ├── Ledger/Ledger.tsx        ← [MODIFICADO] usa useFinancialSummary
│   └── ...
└── test/
    └── setup.ts                 ← [NOVO] config Vitest
```

🔄 Alterações por Fase

✅ Fase 1 — Estabilidade e Segurança

`src/components/ErrorBoundary.tsx` **— MODIFICADO**

- Interface dark mode com ícone de alerta
- Dois botões: **Recarregar página** e **Limpar dados e recarregar**
- Exibe detalhes técnicos do erro (colapsável)

- Removido `localStorage.clear()` automático (era destrutivo)

`src/context/ToastContext.tsx` — NOVO

- Context global para notificações tipo toast
- Métodos: `toast.success()`, `toast.error()`, `toast.info()`, `toast.warning()`
- Duração configurável, empilhamento de toasts

`src/components/ui/ToastContainer.tsx` — NOVO

- Renderiza toasts no canto superior direito
- Animação de entrada e saída
- Botão de fechar manual

`supabase/rls_audit.sql` — NOVO

- Script SQL de auditoria e criação de políticas RLS
- Cobre todas as tabelas: `ledger_transactions`, `fixed_expenses`, `credit_cards`, `credit_transactions`, `shopping_lists`, `car_fuel`, `car_maintenance`, `recurring_bills`, `product_catalog`
- Política: `user_id = auth.uid()` em todas as operações

✅ Fase 2 — Arquitetura e Manutenibilidade

`src/lib/financialCalculations.ts` — NOVO

- Função pura `calculateFinancialSummary()` extraída do hook
- Sem dependências React — testável diretamente
- Regras: exclui virtual-invoices, usa `payment_date`, separa pendentes físicos de virtuais

`src/hooks/useFinancialSummary.ts` — MODIFICADO (refatorado)

- Agora é um wrapper `useMemo` em cima de `calculateFinancialSummary`
- Mesma API pública — zero breaking changes nos componentes

`src/pages/Dashboard.tsx` — MODIFICADO

- Remove ~55 linhas de lógica de cálculo duplicada
- Passa a usar `useFinancialSummary` como fonte única de verdade

`src/pages/Ledger/Ledger.tsx` — MODIFICADO

- Remove `useMemo goalData` de ~75 linhas
- Passa a usar `useFinancialSummary`

`src/lib/utils.ts` — MODIFICADO

- Adicionada função `getDueStatus(dueDate, paid)` → `'paid' | 'overdue' | 'today' | 'upcoming'`
- Garante lógica de status padronizada em todo o app

`src/context/AppContext.tsx` — REFATORADO

- **891 linhas → ~291 linhas** (67% de redução)
- Agora é um orquestrador: gerencia estado e delega lógica para os slices
- API pública `useApp()` **100% preservada**

`src/context/slices/` — NOVO (5 arquivos)

Slice	Responsabilidade	Cross-dependência

`ledgerSlice.ts`	CRUD de transações do caixa	—
`carSlice.ts`	Combustível + manutenções	→ `setLedger`
`shoppingSlice.ts`	Listas, itens, catálogo	→ `setLedger`
`billsSlice.ts`	Contas fixas, recorrentes, faturas	→ `setLedger`, `setCreditTransactions`
`cardsSlice.ts`	Cartões, transações, parcelamentos	—

`.env.example` — NOVO

```
VITE_SUPABASE_URL=https://your-project.supabase.co
VITE_SUPABASE_ANON_KEY=your-anon-key-here
```

✅ Fase 3 — Qualidade e Validação

`src/lib/financialCalculations.ts` + `src/hooks/__tests__/financialCalculations.test.ts` — NOVO

17 testes unitários cobrindo:

- Saldo de abertura acumulado de meses anteriores
- Dupla-contagem de faturas virtuais **completamente bloqueada**
- `payment_date` (não `due_date`) como referência de pagamento
- Pendências físicas: apenas do mês atual; virtuais: sempre
- `projectedBalance = currentBalance - fixedPending`
- Metas: percentage, isSafe
- Dashboard e Ledger produzem **exatamente os mesmos números**
- Saldo negativo tratado corretamente

`src/schemas/` — NOVO (3 schemas + 23 testes)

`ledger.schema.ts` — Lançamentos no Caixa

- `type` : income | expense
- `value` : > 0, máx R$ 1.000.000
- `description` : 2-120 chars
- `payment_method` : dinheiro | pix | débito | crédito | transferência
- `date` : formato YYYY-MM-DD

`fixedExpense.schema.ts` — Contas Fixas + Recorrentes

- `value` : > 0, máx R$ 100.000
- `due_date` : formato YYYY-MM-DD
- `due_day` (recorrente): 1-31, inteiro
- `description` : 2-100 chars

`creditCard.schema.ts` — Cartões + Compras

- `limit_amount` : > 0, máx R$ 500.000
- `closing_day` ≠ `due_day` (validação cruzada com `.refine()`)
- `installments` : 1-48, inteiro
- `status` : open_invoice | closed_invoice | paid

`vite.config.ts` **— MODIFICADO**

- Configuração Vitest: `environment: 'jsdom'`, `globals: true`, `setupFiles`

`package.json` **— MODIFICADO**

- Adicionados scripts: `"test": "vitest"`, `"test:run": "vitest run"`
- Novas devDependencies: `vitest`, `@testing-library/react`, `@testing-library/jest-dom`, `jsdom`
- Nova dependency: `zod`

📊 Git — Histórico de Checkpoints

```
a1b4952  checkpoint: Fases 1 e 2 concluidas  ← PONTO DE RESTAURAÇÃO PRINCIPAL
1b5fccc  fase-3: Vitest + Zod - 40 testes passando
```

Para restaurar para um ponto anterior:

```
git checkout a1b4952   # Restaura para fim das Fases 1+2
git tag                 # Lista tags disponíveis
git checkout fase-2-completa  # Restaura via tag
```

🧪 Suíte de Testes

```
npm run test:run   # Executar 1x (CI/CD)
npm run test       # Modo watch (desenvolvimento)
```

Resultado atual: `40/40` testes ✅ em `~1.25s`

⏭️ Fase 4 — Próximos Passos (pendentes)

Item	Descrição
4.1 PWA	App instalável no celular, caching offline
4.2 Relatórios	Gráficos recharts: linha de saldo, barras, pizza por categoria
4.3 Orçamento	Limite by categoria com barra de progresso
4.4 Import CSV	Upload de extrato bancário com mapeamento automático
4.5 Notificações	Alertas nativos para contas vencendo em 2 dias